

LoRA: Low-Rank Adaptation 低秩适配

LoRA 原论文的思想是：冻结预训练模型权重，在 Transformer 层中注入可训练的低秩分解矩阵，从而减少下游任务的可训练参数量。原论文报告，在 GPT-3 175B 场景中，LoRA 相比使用 Adam 的全参数微调，可训练参数量可减少约 10,000 倍，GPU 显存需求可减少约 3 倍，并且不像部分 Adapter 方法那样增加推理延迟。

(<https://arxiv.org/pdf/2106.09685>)

假设预训练模型参数为 θ_0 ，下游任务数据集为 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ ，模型输出为 $f_\theta(x_i)$ ，损失函数为 $\mathcal{L}(f_\theta(x_i), y_i)$ ，全参数微调的目标是：

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i)$$

也就是说，全参数微调会直接修改模型中的所有参数 $\theta_0 \rightarrow \theta^*$ ，对于大模型来说，这会带来巨大的训练成本、显存成本和存储成本。

Transformer 里有大量线性层。我们先只看一个线性层。设输入为 $X \in \mathbb{R}^{n \times d_{in}}$ ，权重为 $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$ ，输出为 $Y = XW_0^T$ ，全参数微调会把原来的权重 W_0 变成新的权重 W' ，我们可以写成 $W' = W_0 + \Delta W$ ，其中 $\Delta W = W' - W_0$ ，表示下游任务对原模型权重的修改量。于是微调后的输出为 $Y = X(W')^T$

代入 $Y = X(W_0 + \Delta W)^T$ ，展开 $Y = XW_0^T + X(\Delta W)^T$ ，所以全参数微调的是学习一个完整的权重增量 ΔW

对于 $\Delta W \in \mathbb{R}^{d_{out} \times d_{in}}$ ，如果 $d_{out} = 4096, d_{in} = 4096$ ，那么一个矩阵就有：

$$4096 \times 4096 = 16,777,216$$

个参数。这还只是一个线性层里的一个权重矩阵。（LoRA 原论文将大模型微调问题表述为冻结预训练权重并学习低秩增量，以减少全参数微调的成本。）

LoRA 的假设是下游任务需要的权重更新 ΔW 可能具有低秩结构。也就是说，虽然 ΔW 表面上是一个很大的矩阵，但它真正有效的变化方向可能不多。如果一个矩阵的有效变化方向很少，那么它就可以用低秩矩阵近似。LoRA 原论文中也研究了语言模型适配中的 rank-deficiency，即任务适配更新可能并不需要完整高秩空间。

全参数微调训练的是 $\Delta W \in \mathbb{R}^{d_{out} \times d_{in}}$ ，LoRA 不直接训练这个完整矩阵，而是令 $\Delta W = BA$ ，其中 $B \in \mathbb{R}^{d_{out} \times r}$ ； $A \in \mathbb{R}^{r \times d_{in}}$ ，所以 $BA \in \mathbb{R}^{d_{out} \times d_{in}}$ ，维度正好和 ΔW 一致。加入缩放项后，LoRA 的完整形式是 $W' = W_0 + \frac{\alpha}{r} BA$ ，其中 r 是 LoRA rank； α 是 LoRA scaling factor。

设 $M \in \mathbb{R}^{m \times n}$ ，矩阵的秩为 $\text{rank}(M)$ ，表示矩阵中线性无关行或列的最大数量。如果 $\text{rank}(M) = r$ 并且 $r \ll \min(m, n)$ ，那么 M 就是低秩矩阵。LoRA 中 $\Delta W = BA$ ，其中 $B \in \mathbb{R}^{d_{out} \times r}$ ； $A \in \mathbb{R}^{r \times d_{in}}$ ，因为 B 只有 r 列，所以 $\text{rank}(B) \leq r$ ；因为 A 只有 r 行，

所以 $\text{rank}(A) \leq r$ ，矩阵乘法有性质 $\text{rank}(BA) \leq \min(\text{rank}(B), \text{rank}(A))$ 。

因此 $\text{rank}(BA) \leq r$ ，所以 $\text{rank}(\Delta W) = \text{rank}(BA) \leq r$ ，这说明 LoRA 学到的权重增量天然被限制在一个秩不超过 r 的空间里。

完整权重增量为 $\Delta W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ ，参数量为 $N_{\text{full}} = d_{\text{out}} d_{\text{in}}$ ，LoRA 训练两个矩阵 $B \in \mathbb{R}^{d_{\text{out}} \times r}$ ； $A \in \mathbb{R}^{r \times d_{\text{in}}}$ ，所以参数量为 $N_{\text{LoRA}} = d_{\text{out}} r + r d_{\text{in}}$ ，整理 $N_{\text{LoRA}} = r(d_{\text{out}} + d_{\text{in}})$ 。

参数比例为 $\frac{N_{\text{LoRA}}}{N_{\text{full}}} = \frac{r(d_{\text{out}} + d_{\text{in}})}{d_{\text{out}} d_{\text{in}}}$ ，如果 $d_{\text{out}} = d_{\text{in}} = d$ ，那么：

$$\frac{N_{\text{LoRA}}}{N_{\text{full}}} = \frac{r(d + d)}{d^2} = \frac{2rd}{d^2} = \frac{2r}{d}$$

[举例：

$$d = 4096, r = 8, \text{ 则 } \frac{2r}{d} = \frac{16}{4096} = 0.00390625$$

也就是说，一个 4096×4096 的线性层，全参数微调需要训练 16,777,216 个参数，而 LoRA 只需要训练 $4096 \times 8 + 8 \times 4096 = 65,536$ 个参数。]

原始线性层为 $Y = XW_0^T$ ，LoRA 令 $W' = W_0 + \frac{\alpha}{r} BA$ ，代入：

$$Y = X \left(W_0 + \frac{\alpha}{r} BA \right)^T$$

利用转置性质 $(M + N)^T = M^T + N^T$ ，得到 $Y = XW_0^T + \frac{\alpha}{r} X(BA)^T$ ，又因为 $(BA)^T = A^T B^T$ ，所以 $Y = XW_0^T + \frac{\alpha}{r} XA^T B^T$ ，因此 LoRA 线性层的前向传播为 $Y = XW_0^T + \frac{\alpha}{r} XA^T B^T$ ，其中 XW_0^T 是原模型冻结分支， $\frac{\alpha}{r} XA^T B^T$ 是 LoRA 低秩增量分支。

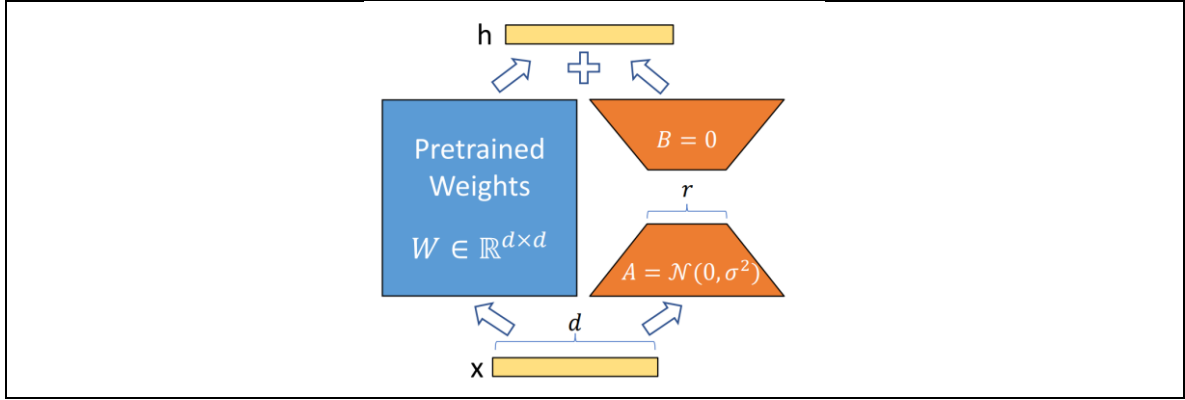
设 $s = \frac{\alpha}{r}$ ，LoRA 层输出为 $Y = XW_0^T + sXA^T B^T$

损失函数为 $\mathcal{L}(Y)$

设 $E = \frac{\partial \mathcal{L}}{\partial Y}$ ，其中 $E \in \mathbb{R}^{n \times d_{\text{out}}}$ ，因为 W_0 被冻结，所以 $\frac{\partial \mathcal{L}}{\partial W_0} = 0$ ，训练时只更新 A 和 B 。LoRA 部分为 $Y_{\text{LoRA}} = sXA^T B^T$ ，令 $M = XA^T$ ，则 $M \in \mathbb{R}^{n \times r}$ ，于是 $Y_{\text{LoRA}} = sMB^T$ ，对 B 求导得 $\frac{\partial \mathcal{L}}{\partial B} = sE^T M$ ，代入 $M = XA^T$ ，得到 $\frac{\partial \mathcal{L}}{\partial B} = sE^T XA^T$ 。

还是从 $Y_{\text{LoRA}} = sXA^T B^T$ 开始。我们使用矩阵微分 $dY = sX(dA)^T B^T$ ，损失微分 $d\mathcal{L} = \text{tr}(E^T dY)$ ，代入 $d\mathcal{L} = \text{tr}(E^T sX(dA)^T B^T)$ ，提出 s ： $d\mathcal{L} = s \text{tr}(E^T X(dA)^T B^T)$ ，利用迹的循环性质 $\text{tr}(ABCD) = \text{tr}(DABC)$ ，得到 $d\mathcal{L} = s \text{tr}(B^T E^T X(dA)^T)$ ，再利用

$tr(P(dA)^T) = tr(P^T dA)$, 所以 $d\mathcal{L} = str((B^T E^T X)^T dA)$, 因此 $\frac{\partial \mathcal{L}}{\partial A} = sB^T E^T X$ 。



LoRA 希望训练刚开始时，模型行为和原模型完全一致。也就是希望 $\Delta W = 0$ ，常见初始化是 $A \sim \mathcal{N}(0, \sigma^2); B = 0$ 。于是 $BA = 0$ ，因此 $W' = W_0$ ，但是这里有一个有意思的梯度现象。前面推导过 $\frac{\partial \mathcal{L}}{\partial A} = sB^T E^T X$ ，如果初始 $B = 0$ ，那么第一步 $\frac{\partial \mathcal{L}}{\partial A} = 0$ ，但是 $\frac{\partial \mathcal{L}}{\partial B} = sE^T X A^T$ ，由于 A 是随机非零矩阵，所以一般有 $\frac{\partial \mathcal{L}}{\partial B} \neq 0$ 。因此训练第一步会先更新 B 。当 B 更新后， A 的梯度也会变成非零。

r 控制 LoRA 增量矩阵的最大秩 $rank(BA) \leq r$ ，如果 $r = 1$ ，说明 LoRA 只能学习非常有限的更新方向。如果 $r = 64$ 说明表达能力更强，但训练参数量也更大。LoRA 参数量为 $r(d_{\text{out}} + d_{\text{in}})$ ，因此 r 越大，参数量越大； r 越小，参数量越小。

LoRA 使用 $\frac{\alpha}{r}$ 来缩放低秩增量 $W' = W_0 + \frac{\alpha}{r} BA$ ，原因是 BA 中每个元素为：

$$(BA)_{ij} = \sum_{k=1}^r B_{ik} A_{kj}$$

当 r 增大时，求和项数量增加，更新量的尺度可能变大。所以用 $\frac{1}{r}$ 进行归一化，

再用 α 控制 LoRA 分支的整体强度。可以把 $\frac{\alpha}{r}$

Transformer 中常见线性矩阵包括 W_Q, W_K, W_V, W_O ，它们分别用于生成：

$$Q = XW_Q; K = XW_K; V = XW_V$$

注意力计算为 $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

然后经过输出投影 $O = Attention(Q, K, V)W_O$

LoRA 可以加到这些矩阵上，例如 $W_Q' = W_Q + \frac{\alpha}{r} B_Q A_Q; W_V' = W_V + \frac{\alpha}{r} B_V A_V$

对于任意矩阵 $\Delta W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ ，如果它的秩为 r ，则可以写成紧凑 SVD:

$$\Delta W = U_r \Sigma_r V_r^T$$

其中 $U_r \in \mathbb{R}^{d_{\text{out}} \times r}$; $\Sigma_r \in \mathbb{R}^{r \times r}$; $V_r^T \in \mathbb{R}^{r \times d_{\text{in}}}$ ，如果令 $B = U_r \Sigma_r$; $A = V_r^T$ ，也就是 $BA = \Delta W$ ，所以，任意秩为 r 的矩阵都可以写成 BA 的形式。

注意，LoRA 通常不是先得到一个完整 ΔW 然后再对它做 SVD 压缩。LoRA 是直接把 ΔW 参数化为 BA ，然后通过梯度下降学习 A, B 也就是说，LoRA 从训练一开始就把搜索空间限制在低秩空间中。

AdaLoRA 进一步把增量更新参数化为类似奇异值分解的形式，并根据重要性分数自适应分配不同权重矩阵的参数预算，这说明 SVD/低秩思想是 LoRA 系列方法的重要理论基础之一。（<https://arxiv.org/pdf/2303.10512>）

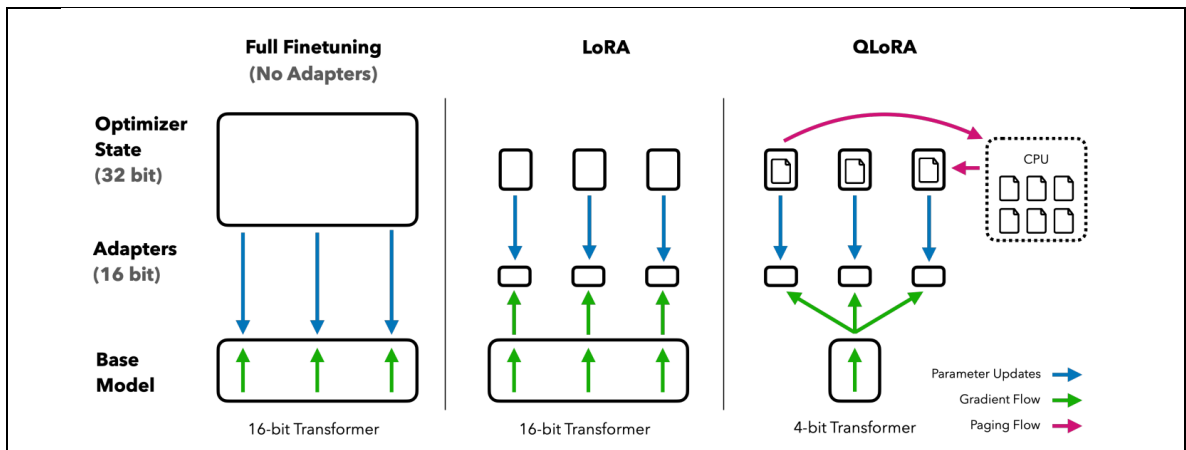
对于语言模型，常见训练目标是自回归语言建模。给定 token 序列 $x_1, x_2, \dots, x_T$ ，模型学习 $p_\theta(x_t | x_{<t})$ ，最大似然目标为 $\max_\theta \prod_{t=1}^T p_\theta(x_t | x_{<t})$

取对数 $\max_\theta \sum_{t=1}^T \log p_\theta(x_t | x_{<t})$ 等价于最小化负对数似然 $\min_\theta \sum_{t=1}^T \log p_\theta(x_t | x_{<t})$ ，

也就是交叉熵损失 $\mathcal{L}(\theta) = \sum_{t=1}^T \log p_\theta(x_t | x_{<t})$ ，全参数微调是 $\min_\theta \mathcal{L}(\theta)$ ，LoRA 微调是 $\min_{A,B} \mathcal{L}(\theta_0, A, B)$ ，其中 θ_0 被冻结。因此 LoRA 不改变损失函数，改变的是可训练参数的形式。

LoRA 主要解决可训练参数太多的问题，QLoRA 进一步解决基础模型本身占用显存太大的问题。QLoRA 的做法是：把基础模型量化到 4-bit，并冻结这个量化后的基础模型，然后把梯度通过冻结的 4-bit 量化模型反传到 LoRA adapter 中。QLoRA 论文提出了 NF4、Double Quantization 和 Paged Optimizers 等技术，并报告可以在单张 48GB GPU 上微调 65B 模型，同时保持接近 16-bit 微调的效果。

可以写成 $Y = X \text{Dequant}(W_q)^T + \frac{\alpha}{r} X A^T B^T$ ，其中 W_q 是量化后的基础模型权重， $\text{Dequant}(W_q)$ 表示反量化后的近似权重。（<https://arxiv.org/pdf/2305.14314>）



标准 LoRA 通常给不同层设置相同或人工指定的 rank 。但是不同层的重要性可能不同。有些层对下游任务更关键，需要更大的 rank ；有些层不重要，可以分配更小的 rank 。AdaLoRA 的思想是根据权重矩阵的重要性，自适应分配低秩预算。

可以理解为普通 LoRA: $\Delta W_l = B_l A_l$ ，并且很多层使用相同 $r_l = r$ ；AdaLoRA: r_l 可以随着层的重要性动态变化。重要层 $r_l \uparrow$ ；不重要层 $r_l \downarrow$ 。

(<https://arxiv.org/pdf/2303.10512>)

原始 LoRA 通常对 A 和 B 使用相同学习率 $\eta_A = \eta_B$ LoRA+ 提出，对 A 和 B A 和 B 设置不同学习率，可能提升训练效率。LoRA+ 论文从大宽度网络的 scaling argument 出发，指出原始 LoRA 中两个 adapter 矩阵使用相同学习率可能导致大宽度模型中的特征学习效率不佳，并提出通过合适的学习率比例来改善这一点。

可以写成 $A \leftarrow A - \eta_A \frac{\partial \mathcal{L}}{\partial A}$; $B \leftarrow B - \eta_B \frac{\partial \mathcal{L}}{\partial B}$ ，其中 $\eta_A \neq \eta_B$ 。

(<https://arxiv.org/pdf/2402.12354>)

	LoRA	LoRA+
Parameterization	Pretrained Weights $W \in \mathbb{R}^{n \times n}$ $+$ $B \times A$	
Training	$A \leftarrow A - \eta \times G_A$ $B \leftarrow B - \eta \times G_B$	$A \leftarrow A - \eta \times G_A$ $B \leftarrow B - \lambda \eta \times G_B$ $\lambda \gg 1$

LoRA 的优点包括：

1. 可训练参数少。
2. 训练显存低。
3. 每个任务只需要保存一份 LoRA adapter。
4. 可以合并回原模型权重。
5. 对大语言模型、多模态模型、扩散模型等都很实用。
6. 在很多场景中能达到接近全参数微调的效果。

LoRA 也有局限：

1. 如果任务和原模型差异很大，低秩更新可能不够。
2. r 太小会欠拟合。
3. r 太大会增加参数量和过拟合风险。
4. 不同层的重要性不同，统一 rank 不一定最优。
5. LoRA 主要修改线性层，对模型结构本身的能力改变有限。

AdaLoRA、LoRA+、QLoRA 等后续方法分别从自适应 rank 分配、优化学习率、量化显存压缩等方向扩展了原始 LoRA。